



Python Programming Fundamentals

Shop v3.0 — Full CRUD

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. **CRUD Overview**
2. **Create — `add_product()`**
3. **Read — List and Find**
4. **Update — `update_price()`**
5. **Delete — `remove_product()`**
6. **Full Menu Integration**
7. **Summary**



Outline

1. CRUD Overview

2. Create — `add_product()`

3. Read — List and Find

4. Update — `update_price()`

5. Delete — `remove_product()`

6. Full Menu Integration

7. Summary



1.1 Create · Read · Update · Delete

Letter	Operation	Shop method
C	Create	<code>add_product(product)</code>
R	Read	<code>get_all_products()</code> , <code>find_by_name()</code>
U	Update	<code>update_price(name, new_price)</code>
D	Delete	<code>remove_product(name)</code>

CRUD is the foundation of almost every data-driven application.



Outline

1. CRUD Overview
- 2. Create — `add_product()`**
3. Read — List and Find
4. Update — `update_price()`
5. Delete — `remove_product()`
6. Full Menu Integration
7. Summary



2. Create — add_product()

```
# In shop.py
def add_product(self, product: Product):
    # Check for duplicate ID
    for p in self.__products:
        if p.get_product_id() == product.get_product_id():
            raise ValueError(
                f"Product ID {product.get_product_id()} already exists"
            )
    self.__products.append(product)

# In driver.py
def add_product_menu(shop: Shop):
    try:
        name = input("Product name: ").strip()
        price = float(input("Price: €"))
        pid = int(input("Product ID: "))
```



2. Create — `add_product()`

```
shop.add_product(Product(name, price, pid))
print(f"Added successfully!")
except (ValueError, TypeError) as e:
    print(f"Error: {e}")
```



Outline

1. CRUD Overview
2. Create — `add_product()`
- 3. Read — List and Find**
4. Update — `update_price()`
5. Delete — `remove_product()`
6. Full Menu Integration
7. Summary



3. Read — List and Find

```
# In driver.py
def list_all(shop: Shop):
    products = shop.get_all_products()
    if not products:
        print("Shop is empty.")
        return
    print(f"\n{'ID':>5}  {'Name':<20} {'Price':>8}")
    print("-" * 37)
    for p in products:
        print(f"{p.get_product_id():>5}  "
              f"{p.get_name():<20} "
              f"€{p.get_price():>7.2f}")

def find_product_menu(shop: Shop):
    name = input("Search by name: ").strip()
```



3. Read — List and Find

```
p = shop.find_by_name(name)
print(f"Found: {p}" if p else "Not found.")
```



Outline

1. CRUD Overview
2. Create — `add_product()`
3. Read — List and Find
- 4. Update — `update_price()`**
5. Delete — `remove_product()`
6. Full Menu Integration
7. Summary



4. Update — update_price()

```
# In shop.py
def update_price(self, name: str, new_price: float) -> bool:
    p = self.find_by_name(name)
    if p is None:
        return False
    p.set_price(new_price)    # setter validates > 0
    return True

# In driver.py
def update_product_menu(shop: Shop):
    name = input("Product to update: ").strip()
    try:
        new_price = float(input("New price: €"))
        if shop.update_price(name, new_price):
            print("Price updated.")
        else:
```



4. Update — update_price()

```
        print(f"'{name}' not found.")  
except ValueError as e:  
    print(f"Error: {e}")
```



Outline

1. CRUD Overview
2. Create — `add_product()`
3. Read — List and Find
4. Update — `update_price()`
- 5. Delete — `remove_product()`**
6. Full Menu Integration
7. Summary



5. Delete — remove_product()

```
# In shop.py (already have this)
def remove_product(self, name: str) -> bool:
    p = self.find_by_name(name)
    if p:
        self.__products.remove(p)
        return True
    return False

# In driver.py
def remove_product_menu(shop: Shop):
    name = input("Product to remove: ").strip()
    p = shop.find_by_name(name)
    if p is None:
        print(f"'{name}' not found.")
        return
    confirm = input(f"Remove '{p.get_name()}'? (y/n): ")
```



5. Delete — remove_product()

```
if confirm.lower() == "y":  
    shop.remove_product(name)  
    print("Removed.")  
else:  
    print("Cancelled.")
```



Outline

1. CRUD Overview
2. Create — add_product()
3. Read — List and Find
4. Update — update_price()
5. Delete — remove_product()
- 6. Full Menu Integration**
7. Summary



6. Full Menu Integration

```
def show_menu():
    print("\n=== Shop v3.0 ===")
    print("1. Add product")
    print("2. List all products")
    print("3. Find product")
    print("4. Update price")
    print("5. Remove product")
    print("0. Exit")

def main():
    shop = Shop("My Shop")
    while True:
        show_menu()
        match input("Choice: ").strip():
            case "1": add_product_menu(shop)
            case "2": list_all(shop)
```



6. Full Menu Integration

```
case "3": find_product_menu(shop)
case "4": update_product_menu(shop)
case "5": remove_product_menu(shop)
case "0": break
case _:   print("Invalid option.")
```



Outline

1. CRUD Overview
2. Create — `add_product()`
3. Read — List and Find
4. Update — `update_price()`
5. Delete — `remove_product()`
6. Full Menu Integration
- 7. Summary**



7. Summary

CRUD	Python
Create	<code>list.append()</code>
Read	<code>for loop · list.index()</code>
Update	<code>find + modify attribute</code>
Delete	<code>list.remove()</code>

The menu is just the **user interface** — the real logic lives in Shop and Product.



Thanks for Watching - Any questions?

